Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

CHESSVERSE

Team Name: *Team #*	*Checkmates* *DSCPP-III-2026-T175*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Singh, Priyani - 2510019948* *priyanis997@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Kotnala, Arnav - 2510380036* *kotnalaarnav2007@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Gairola, Swastik - 2510380064* *tanmaygairola69@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Riya - 2510376383* *rosieriyaa@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

ChessVerse is developed as an offline chess application created in C/C++ programming language using Raylib graphics library. This application is split into a graphical interface a chess engine and an algorithm decision maker. The interface is responsible for drawing chessboard and the pieces and by recognizing user input. The chess engine is used to hold the board state, evaluate moves, and enforce rules. The algorithm is used by the application to decide which moves to make as an opponent. The engine utilizes Minimax algorithm with Alpha-Beta Pruning implementation to cancel unneeded branches of the trees. The application also features the Hint Engine and the Move Noter components. The chess game is played locally on a user's PC without the need to use any external online services. The difficulty of the game is set by the user at the start of the application according to algorithm modes. The application is developed as a modular system where the graphical interface, chess rules engine, and algorithm decision-maker are separated to provide easier maintenance, debugging, and development. Developers focus on the application's ability to correctly validate moves, update the game state, make decisions based on algorithms, and provide the correct interaction between the graphical interface and the chess engine.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

ChessVerse is designed as an offline chess application created in C/C++ with the help of the Raylib graphics library. It consists of three major parts, including the graphical user interface, chess engine, and algorithm decision-maker. The first one is responsible for creating the chessboard, visualizing the pieces, and processing all user inputs. Chess engine handles the game rules, moves' validation, and game's states, including check, checkmate, castling, and promotion of pawns. The algorithm allows the computer to make moves by choosing the best option using the Minimax approach and Alpha-Beta Pruning to reduce the branching factor. Besides, there are some additional features, such as a Hint Engine, Move Noter, and the opportunity to set the difficulty level of the opponent manually. Thus, it is possible to play an offline chess game and monitor all moves that have been made. The GUI part of the application was created in C++, while the chess engine and algorithm were developed in C/C++. It does not require any external services since it is designed to work offline on a user's PC. It is modular therefore each part can be developed separately. For example, there is an opportunity to work on the chessboard, process user inputs, and validate moves. This way, it is possible to separate the GUI and logic parts of the game and continue working on them separately. Developers currently work on the possibility of making the game more modular and easier to separate. It is their primary task to accomplish in order to make sure that all other functions operate correctly and the game will be successful.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Chessboard GUI, piece display using Raylib; basic chess movement, move validation, backend	Arnav Kotnala
C++ algorithm as opponent using Minimax with Alpha-Beta Pruning and manually selectable difficulty levels, Hint Engine, Move Noter, Backend	Priyani Singh
Minimax + Alpha-Beta algorithm, implementation of linked list for trivia questions, backend	Swastik Gairola
Accounts and login, board, pieces collection	Riya

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

1. Chess Rule Validation validates the moves according to the chess rules and implementing special situations like check, checkmate, castling, pawn promotion etc on different positions.
2. Component Integration: Integrating the graphical interface, algorithm, Hint Engine, and Move Noter in unison to reflect the game board and its state correctly in the user interface.
3. Algorithm Performance and Testing: While Minimax can become exponentially more expensive with higher search depths, Alpha-Beta Pruning can eliminate branches with little to no examination. The team is currently working on debugging individual modules and testing the performance of the game algorithm on different positions. However, there is a need for end-to-end testing.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Minor corrections in UI	Arnav Kotnala
Move Noter and Hints to be adjusted for online games	Priyani Singh
Randomness in code for custom room	Swastik Gairola

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

1. The primary output of my effort is an offline PC chess app written in C/C++ using Raylib with a graphical interface of a chessboard, piece movement, move validation, and special rules implementation.
2. This application features an algorithm versus player game mode along with Minimax algorithm with Alpha-Beta Pruning while manually set difficulty levels, hint engine, and move noting, all individually verified.
3. However, the left tasks are integration tests, end-to-end testing, issue resolution, final build, and documentation elaboration for the project.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

1. The project has progressed beyond the initial design phase and is now fully implemented into a functioning application. The graphical interface, chess movement logic, AI-opponent, Hint Engine, Move Noter, manual difficulty selection mechanism, and special-rule handler have all been implemented and tested with ease using C/C++ and Raylib.
2. Integration and full-game testing remain in progress. The next stage would involve debugging and making sure that the board, game state, AI decisions, hints, and move-records are all consistent with one another while also preparing a stable demonstration build for the next step. Progress is measured in completed features rather than a percentage.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository: `https://github.com/arnavkotnala/chess-pbl-175/tree/main/github_repo`
The repository accommodates the ChessVerse source code and the project itself. The essential components of the project are presented below:
• Chess engine
• Raylib graphical interface
• Algorithm to choose the moves with Minimax and Alpha-Beta Pruning
• Hint Engine
• Move Noter
The link to the repository indicates the main branch; however, the critical commits should be taken from the repository history.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status	Notes
GUI / board	PASS	Board displays correctly.
Piece movement	PASS	Movement verified.
Move validation	PASS	Validation verified.
algorithm / difficulty	PASS	algorithm and manual levels verified.
Hint / Move Noter	IN PROGRESS	To be adjusted for online gameplay.
Special rules	PASS	Rule handling verified.
Module integration	IN PROGRESS	Integration checks remain.
Complete-game test	PENDING	Full-game testing remains.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

1. The implementation and verification of the GUI, chess movement and validation, AI opponent, manually selectable difficulty, the Hint Engine, the Move Noter, and special-rule processing have been completed individually.
2. The application as a whole is still a work in progress, as there are additional tests and debugging to be done. The final products will include the offline chess application, the C/C++ code, the project documentation, and a demonstration.